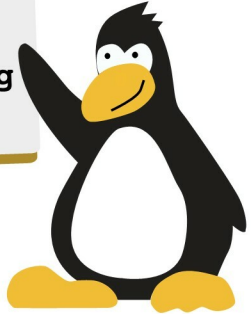


Get Started with Kernel Module Programming



Kernel module programming helps create drivers that are missing in the current kernel's list. This article takes readers through the basics of creating a kernel module, navigating through the output, loading and removing a module. Knowledge of 'C' language (most of the kernel code is in C) and the GNU C extensions is required.

I use the 10.04 32-bit desktop edition, with kernel 2.6.34.12. New distros carry the kernel 3.x series. There is a very slight difference between the architecture of 2.x and 3.x series, so what I discuss in this article will work on both. The source code for this article can be downloaded from http://www.linuxforu.com/article_source_code/oct12/kernel_module.zip

So let's get started with the traditional 'hello world' module; the filename is *hello.c*.

```
#include<linux/init.h>           #1
#include<linux/module.h>          #2

MODULE_LICENSE("GPL");            #3

static int hello_init(void){      #4
    printk(KERN_ALERT "Hello world"); #5
    return 0;
}
```

```
static void hello_exit(void) {    #6
    printk(KERN_ALERT "GoodBye");
}

module_init(hello_init);         #7
module_exit(hello_exit);         #8
```



Note: Most libraries are in the include folder of the kernel root directory. In module programming as well as in core kernel programming, `<linux/headerfile>` is short for `/root kernel directory/include/linux/headerfile`. So you can easily say that *include* is the base directory for simple libraries. For architecture-related libraries, *arch* is the base directory.

Now let me explain the code, line by line.

1. *init.h* is included because it will get executed as soon as you load the module in the kernel space. It is mandatory to include this header file in almost every program.
2. All the module functions and recognisers are present in